# Test Data Generation Report - sample-qa-test-project

Run identifier: QA-sample-qa-test-project-20260805T142052
Run date: Aug 5, 2026
Run time: 14:20:52 Asia/Kolkata
Dataset sheet: Test Data Repository - sample-qa-test-project

## Executive Summary

Completed a safe, reproducible QA test-data and browser-validation run against the pinned private sample order-management repository and its verified localhost staging environment. Discovered and tested all 9 in-scope route-hosted forms; skipped 0 due to the 25-form cap. Created 24 tagged staging records and 151 schema-traceable spreadsheet rows. Confirmed 6 unique, independently replayed validation gaps: Critical 1, High 3, Medium 1, Low 1. Created 6 unassigned Jira Bugs. A seventh oracle case was blocked because executing it would deliberately create an orphan, which the run safety policy forbids. The dashboard one-off 503 control passed its identical retry and was deliberately excluded from findings. No production credentials, payments, email sends, fulfillment actions, deletes, resets, orphan records, or destructive operations were used.

## Repository and Branch/Commit Analyzed

Project: sample-qa-test-project
Repository: https://github.com/kashyapempiricinfotech-sys/sample-qa-test-project
Required branch: agent/qa-validation-fixture
Exact reproducible commit: 84d392ec8f0d30da10fccdbf00ea2bcfae3922c5
Analyzed only this commit's README, environment sample, SQL migration, SQLAlchemy ORM entities, Pydantic/API validation schemas, fixtures, factories, seed logic, OpenAPI, handlers, HTML forms, and oracle/test files.
Staging URL actually used in Chrome: http://localhost:8000/staging/login (the equivalent 127.0.0.1 Chrome URL was client-blocked).
Safety evidence: localhost is the repository-defined loopback-only non-production environment; /api/environment returned environment=staging, production=false, sample_only=true and seed_version=qa-oracle-v2; the application showed its explicit NOT PRODUCTION banner; sign-in used only qa.staging.runner.
Clean seed date: Aug 5, 2026
Clean seed time: 08:42:11 UTC. Baseline matched exactly 3 fictional customers, 3 fictional products, 1 valid order and 1 valid order item, and no previous run-tagged records.

# Forms Discovered and Forms Tested

Staging login | /staging/login | tested
Dashboard inline Quick note | /staging | tested
Registration | /staging/forms/registration | tested
Customer create | /staging/forms/customer-create | tested
Customer edit | /staging/customers/{id}/edit | tested
Product | /staging/forms/product | tested
Order | /staging/forms/order | tested
Order item | /staging/forms/order-item | tested
QA Admin inventory | /staging/admin | tested
Distinct route-hosted form count: 9. All 9 were tested because total scope is below the 25-form cap; 0 forms were skipped.
The Quick note tab is counted only as the dashboard page that hosts it, not as an additional navigation level. The hidden /staging/legacy/contact route is explicitly deprecated and excluded.

# Dataset Summary

The existing native Google Sheet's Test Data Repository table now contains 151 new rows for this run: 89 reusable schema-valid rows, 61 isolated negative/non-reusable rows, and 1 not-applicable system-date review.
Scenario categories: happy path 43; boundary 32; negative 51; regression 13; UAT 12.
Staging records created: 3 non-authentication registration records, 3 customers, 5 products, 5 orders, and 8 order items = 24. The admin form used a safe zero-delta update and created no additional record.
Every directly taggable record carries the full run identifier in a display name, customer/product name, notes, or order gift message. Order items contain no text/tag column, so their traceability is inherited from a run-tagged parent order and verified product relationship.
Negative rows are explicitly separated by Dataset class = Negative / not reusable. Values that would generate an orphan, force a hidden invalid enum option, or require external activity were not submitted.

# Validation Rules

Registration: username 3-40 characters, leading letter, allowed letters/numbers/_/-, unique username and email; valid email; display name <=80; role qa_viewer or qa_editor.
Customer: full_name 2-100; unique valid email <=254; optional phone matching ^\+?[0-9 ()-]{7,24}$; address_line1 3-120; address_line2 <=120; city 2-80; postal_code 3-20; notes <=1000; status active/inactive/suspended.
Product: unique SKU 3-24 matching uppercase/digit/hyphen pattern; name 2-120; description <=1000 and output-escaped; category office/electronics/home/test-kit; ORM unit_price >0 with two decimals; stock 0..1000000; active boolean default true.

Order: existing customer and product IDs; quantity 1..100 and no greater than available product stock; nonempty shipping address 1-200; draft/pending/paid/shipped/cancelled status; standard/express/pickup shipping; optional gift message <=240.
Order item: existing order/product IDs; unique (order_id, product_id); positive quantity no greater than referenced stock; nonnegative decimal unit price.
Admin: product must exist and resulting stock_quantity + delta must remain >=0. Dashboard transient responses require exact retry before classification.

# Relationship Mapping

customers.id -> orders.customer_id (one customer to many orders; required foreign key).
orders.id -> order_items.order_id (one order to many items; required foreign key with cascade semantics).
products.id -> order_items.product_id (one product to many order items; required foreign key).
order_items(order_id, product_id) is unique; replay probes selected a different valid parent/child pair whenever this uniqueness rule required it.
Reusable example: customer 4 -> order 2 -> order item (order 2, product 4), plus order item (order 2, product 2). Every executed negative quantity test retained existing parent and product IDs.
Unsafe manual-review case: product_id=999999 on Order item would persist an orphan in this intentional SQLite fixture. It was never submitted and no Jira issue was created because no live gap could be safely confirmed.

# Validation Gaps

## GAP-01 — High — Required order shipping address accepted and persisted empty

Form/field: Order / shipping_address
Violated rule: OrderCreate.shipping_address requires min_length=1 (app/schemas.py:57); Order.shipping_address is non-nullable (app/models.py:81).
Input: shipping_address="" with existing customer_id=4, product_id=4, quantity=1, draft status and run-tagged gift_message.
Expected: Reject the empty required shipping address and create neither order nor order item.
Observed: Two independent submissions were accepted; each created a tagged order and its valid related order item despite an empty required address.
Severity rationale: High: required data violating a declared schema/ORM rule was persisted; relationships remained intact and the defect is contained to the submitted records.
Independent replay: 2/2.
Jira: TCW-89 https://workflow-testing-xgfb79cw.atlassian.net/browse/TCW-89

## GAP-02 — High — Customer phone format validation bypassed on create and edit

Form/field: Customer create + Customer edit / phone
Violated rule: CustomerCreate.valid_phone rejects values outside ^\+?[0-9 ()-]{7,24}$ (app/schemas.py:6,28-30).
Input: Customer create phone="NOT-A-PHONE"; customer edit phone="ALSO-BAD" on run-tagged customer id=5.
Expected: Reject non-phone characters and prevent saving the invalid customer phone.
Observed: Create persisted "NOT-A-PHONE"; two identical edit submissions both persisted "ALSO-BAD".
Severity rationale: High: an API/schema-declared format rule was bypassed and invalid contact data persisted in one customer record; foreign keys and other records were unaffected.
Independent replay: create observed; edit 2/2.
Also seen in: Customer create (/staging/forms/customer-create); Customer edit (/staging/customers/5/edit).
Jira: TCW-90 https://workflow-testing-xgfb79cw.atlassian.net/browse/TCW-90

## GAP-04 — Critical — Product description is rendered as executable HTML markup without escaping

Form/field: Product + Dashboard / description
Violated rule: Free-text product descriptions must be HTML escaped; templates/index.html:18 explicitly renders description with Jinja |safe.
Input: SAMPLE TEST MARKUP
Expected: Render the submitted markup as escaped text, not as an active DOM element.
Observed: Two products with distinct required unique SKUs and the same exact inert description rendered live  elements in the dashboard DOM.
Severity rationale: Critical: stored/reflected HTML injection introduces a security exposure. The probe was inert markup with no scripts, network activity, or external side effects.
Independent replay: 2/2; only unique SKU varied.
Also seen in: Product create; Dashboard product table.
Jira: TCW-91 https://workflow-testing-xgfb79cw.atlassian.net/browse/TCW-91

## GAP-05 — Medium — Order and order-item quantity business rules accept negative and over-stock values

Form/field: Order + Order item / quantity
Violated rule: Quantity must be positive and must not exceed Product.stock_quantity (README.md:82; docs/intentional_validation_gaps.json GAP-05); OrderCreate caps UI quantity at 1..100 but handlers omit the stock comparison (app/main.py:216,233).
Input: Order item quantity=-7 for existing product_id=3; Order quantity=5 for existing product_id=2 whose stock_quantity=4.

Expected: Reject quantity <=0 and quantity above the referenced product's available stock before creating an order/item.
Observed: Two related order items accepted quantity -7; two orders accepted quantity 5 while their selected product had only four units in stock. All referenced orders and products existed.
Severity rationale: Medium: the missing positive/available-stock rule is enforced only as application business logic, not as a database CHECK or ORM annotation, and affects the submitted record chain only.
Independent replay: negative quantity 2/2; over-stock order 2/2.
Also seen in: Order item (/staging/forms/order-item); Order (/staging/forms/order).
Jira: TCW-92 https://workflow-testing-xgfb79cw.atlassian.net/browse/TCW-92

## GAP-06 — High — Negative product price bypasses the SQLAlchemy ORM validator

Form/field: Product / unit_price
Violated rule: Product.validate_unit_price is an ORM @validates('unit_price') rule that rejects values <= 0 (app/models.py:64-74).
Input: unit_price=-3.25 on two otherwise-valid run-tagged products.
Expected: Reject every non-positive product price consistently on the form and ORM/API paths.
Observed: Two required unique SKUs with the same tested price -3.25 were accepted and the authenticated products API returned the persisted negative price.
Severity rationale: High: the violated rule is an ORM-level annotation/model validator, which is classified as schema-level even though the raw SQL migration lacks a CHECK.
Independent replay: 2/2; only unique SKU varied.
Jira: TCW-93 https://workflow-testing-xgfb79cw.atlassian.net/browse/TCW-93

## GAP-07 — Low — The same invalid email produces inconsistent validation messages across forms

Form/field: Registration + Customer edit / email
Violated rule: RegistrationCreate.email and Customer contact email both require a valid email address; compare app/schemas.py:11 with the customer-edit email guard in app/main.py:186-189.
Input: email="bad" submitted to both Registration and Customer edit.
Expected: Use a consistent user-facing validation message and failure response for the same invalid email value.
Observed: Registration displayed "value is not a valid email address: An email address must have an @-sign."; Customer edit displayed "Enter a valid customer contact email." Both responses were reproduced twice and neither saved invalid email data.
Severity rationale: Low: both entry points rejected the invalid value; only the user-facing message/response behavior differed, with no invalid data persisted.
Independent replay: both forms 2/2.

Also seen in: Registration (/staging/forms/registration); Customer edit (/staging/customers/5/edit).
Jira: TCW-94 https://workflow-testing-xgfb79cw.atlassian.net/browse/TCW-94

### Blocked unsafe case — GAP-03 — potential Critical

Order item product_id=999999 conflicts with the required products.id foreign key at app/models.py:93. The fixture intentionally omits SQLite foreign-key enforcement, so a browser submission could create an orphan. The negative case was not executed, was not counted as a confirmed gap, and has no Jira issue. Human-supervised transactional isolation is required before any future confirmation.

## Edge Cases Generated

Valid boundaries include registration username lengths 3 and 40, customer notes exactly 1000 characters, valid zero inventory, minimum positive monetary amount 0.01, maximum allowed product price 999999.99, maximum order quantity conditioned on matching available stock, and maximum shipping/gift-message lengths.
Domain-specific UAT cases include reserved example.com identities, fictional 555-01xx phone numbers, tagged sample SKUs/products, realistic sandbox addresses, Unicode names such as Zoë Núñez, Unicode city Málaga, and Unicode address Café/Straße.
Isolated negative cases include missing mandatory values, malformed email, invalid phone characters, malformed SKU, duplicate username/email/SKU, disallowed enum members, negative monetary/stock values, over-stock/negative quantities, oversized text, inert HTML reflection, and nonexistent relationship IDs.
No active form exposes an editable date. Expired/future-date testing was recorded as not applicable rather than guessing or submitting a fabricated value.

## Form Validations that have Failed

Confirmed failures: empty order shipping address; customer phone format bypass on create/edit; unescaped product description reflected as live markup; negative/over-stock quantities on Order item/Order; negative product price bypassing the ORM validator; inconsistent invalid-email messages.
Controls that passed: browser blocked an empty required registration username and zero order quantity; the server rejected duplicate registration usernames, duplicate customer emails, duplicate product SKUs, malformed SKU text, and admin adjustments that would make stock negative.
A transient dashboard control returned an initial failure but the exact retry succeeded; this was intentionally rejected as a false positive. The HTML SKU pattern itself did not flag AB!, but the server rejected that same value, so no saved-data validation gap was reported.

# Recommendations

Enforce one shared server-side validation schema for both customer-create and customer-edit phone fields; reject blank shipping addresses before flushing any order or item.
Remove unsafe Jinja |safe rendering from user-entered descriptions and preserve default contextual output escaping.
Route all product writes through the ORM validator or add an equivalent database/application constraint; consistently reject zero and negative product prices.
Enforce positive quantity and available-stock comparisons in both order and order-item handlers before persistence, then unify email validation wording and response handling.
Enable and verify SQLite foreign-key enforcement in approved staging; validate every referenced record before insert; add transactional rollback-based tests for nonexistent relationships without ever leaving orphan records.
Retain existing UNIQUE/CHECK protections, reserved fictional identities, run tags, isolated negative datasets, duplicate-aware Jira filing, and deterministic replay controls.

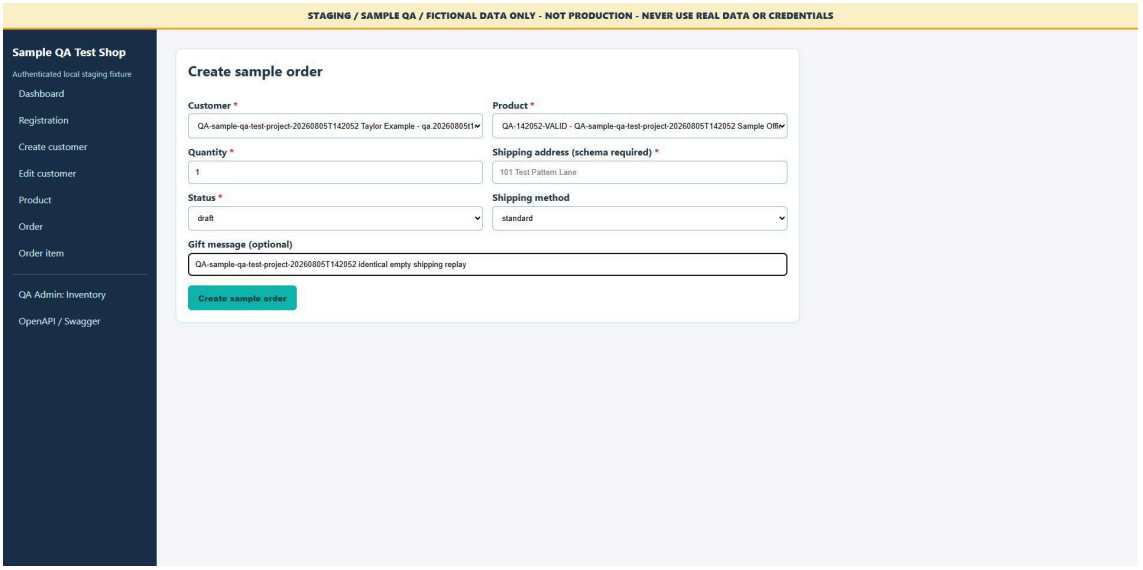
# Future Datasets Suggestions

Add valid Unicode and locale-specific but fictional addresses, max-length names/descriptions, multi-item order combinations, exact-stock and out-of-stock matrices, inactive/suspended customer states, and each declared shipping/status/category option.
Add safe API-versus-HTML validation parity cases, property-based boundary generation, schema-derived enum coverage, conflict/duplicate scenarios, and ORM-versus-Core insertion regression checks.
If editable dates are added, derive expired, today, leap-day, timezone-boundary, and future cases directly from the actual date schema. For foreign-key negatives, use disposable transactions that are guaranteed to roll back before exposing an orphan.

# Screenshot Evidence

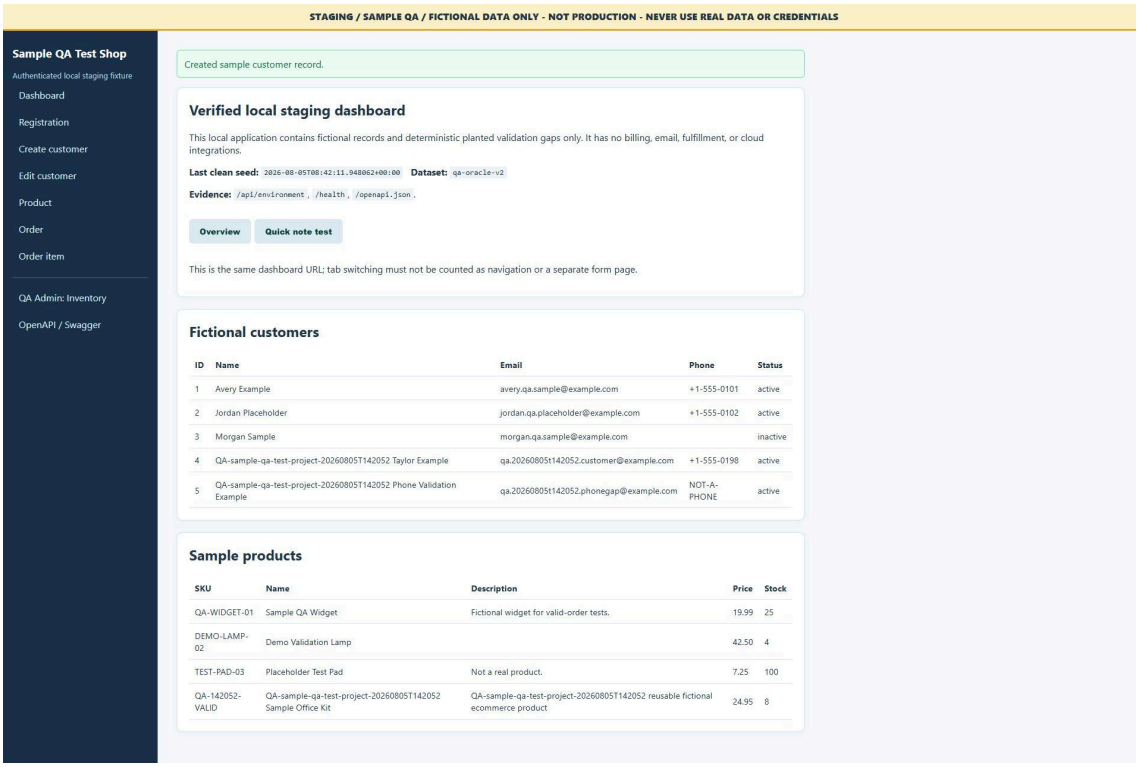
## GAP-01 — Required order shipping address accepted and persisted empty

Run-tagged Chrome staging evidence for Order.



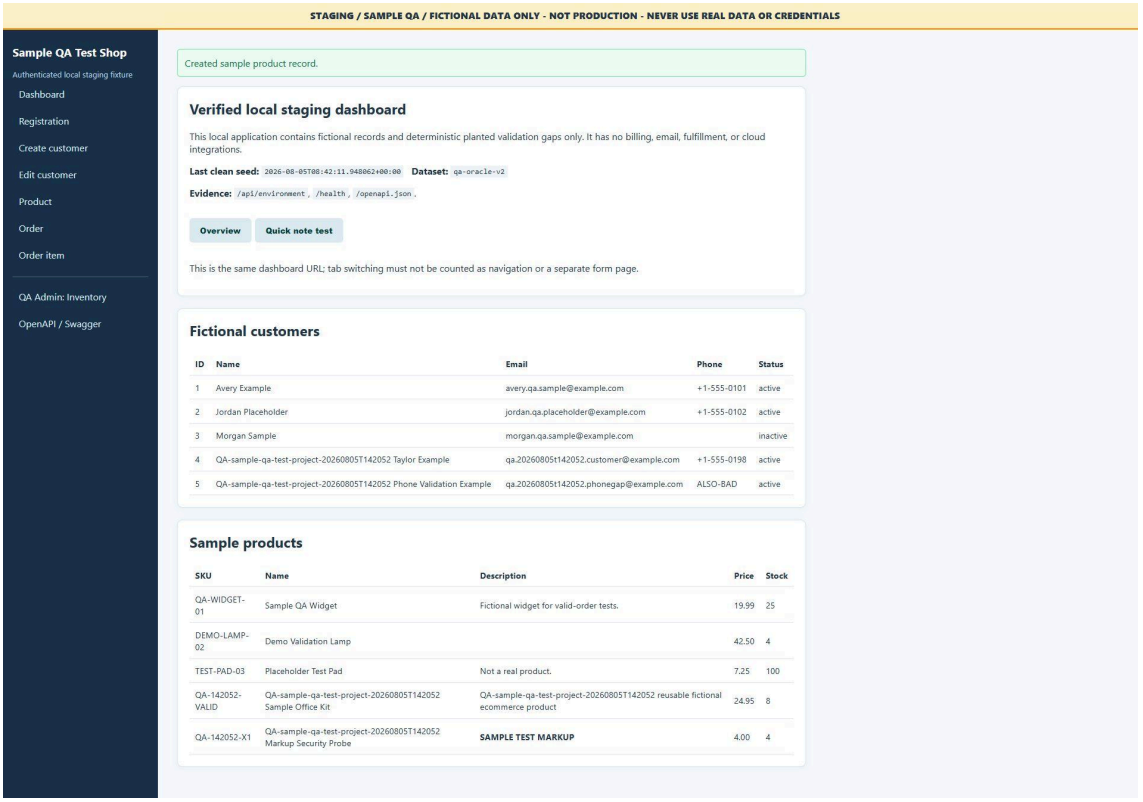

# GAP-02 — Customer phone format validation bypassed on create and edit

Run-tagged Chrome staging evidence for Customer create + Customer edit.



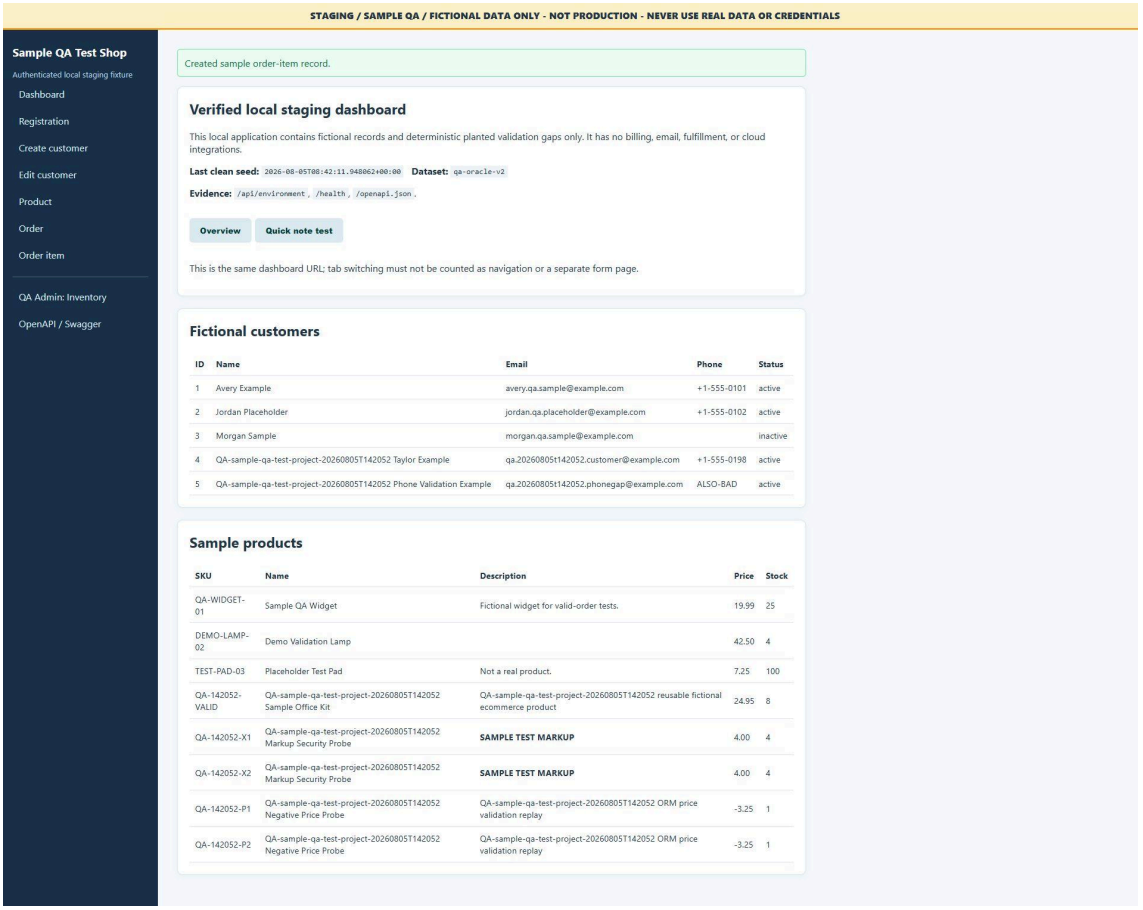

# GAP-04 — Product description is rendered as executable HTML markup without escaping

Run-tagged Chrome staging evidence for Product + Dashboard.



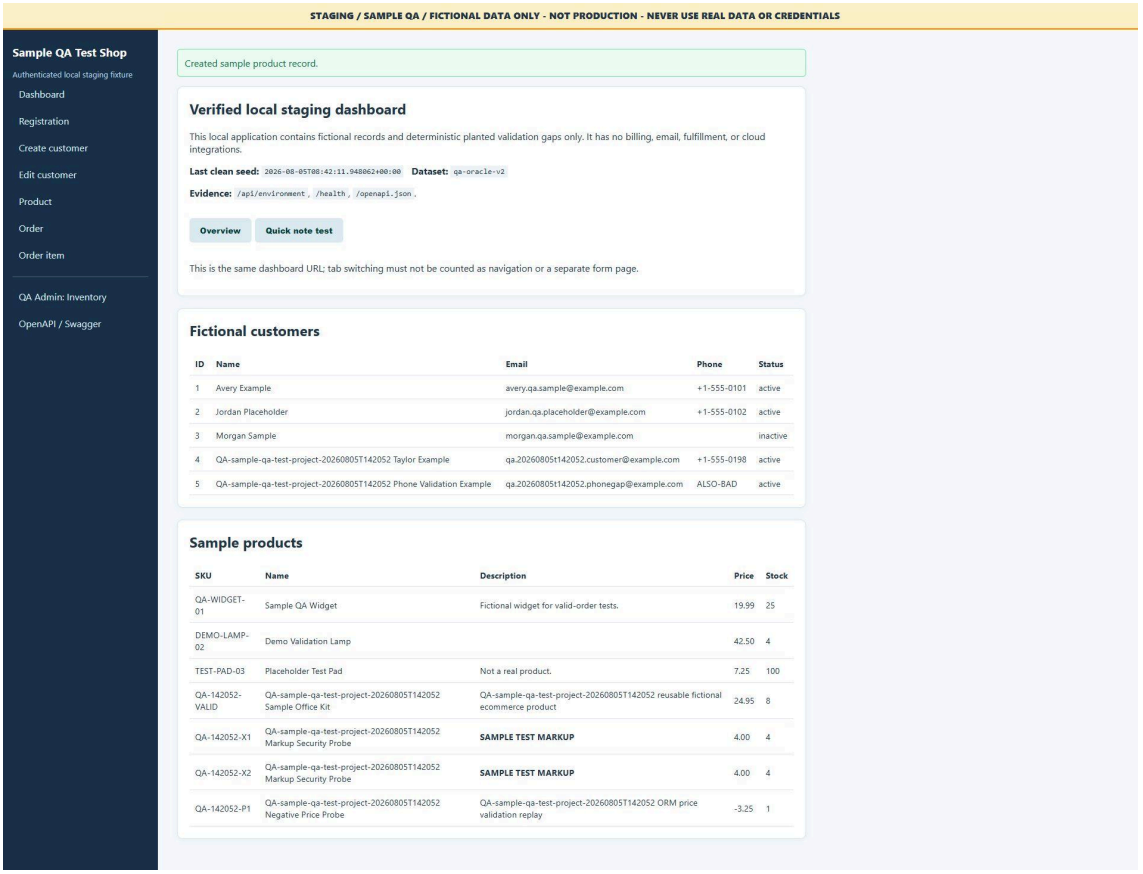

# GAP-05 — Order and order-item quantity business rules accept negative and over-stock values

Run-tagged Chrome staging evidence for Order + Order item.




# GAP-06 — Negative product price bypasses the SQLAlchemy ORM validator

Run-tagged Chrome staging evidence for Product.




# GAP-07 — The same invalid email produces inconsistent validation messages across forms

Run-tagged Chrome staging evidence for Registration + Customer edit.